# Highlights

**NeuPath: A hybrid learning-based optimization approach for emergency search path planning**

Anonymous author

- NeuPath, integrating learning and optimization techniques, is proposed to effectively solve Optimal Search Problem (OSP) for emergency.
- For the first time, the OSP is represented as bipartite graphs, enhancing the extraction of problem features and dimensionality reduction through Graph Neural Networks (GNNs).
- We enhance the GNN with a two-stage aggregation mechanism to improve prediction accuracy and scalability.
- We propose a block-wise trust-region scheme that incrementally fixes variables by block.
- NeuPath achieves an average 2.6× speedup over exact solvers (Gurobi, SCIP) in classical OSP scenarios.

# NeuPath: A hybrid learning-based optimization approach for emergency search path planning

Anonymous author

**Abstract**

Search operations play a vital role in humanitarian emergencies, where maximizing the probability of finding survivors requires highly efficient solutions. A key challenge lies in the limitations of existing methods, which often struggle with high-dimensional constraints and sparse decision spaces, particularly in large-scale scenarios. To address this, we propose NeuPath, a hybrid learning-based optimization framework that accelerates the discovery of high-quality solutions. NeuPath first formulates the optimal search problem (OSP) as a bipartite graph representation to enhance feature extraction and scalability. It then predicts an initial solution using a graph neural network (GNN) augmented with a two-stage aggregation mechanism, followed by refinement via a block-wise trust-region scheme. Extensive experiments on OSP static scenarios (500 instances) demonstrate that NeuPath achieves significant speedups over exact solvers, with performance gains of 2.48× (Gurobi) and 2.74× (SCIP) across varying problem sizes. For large-scale random scenarios (500 instances), the solution quality of this method also significantly exceeds that of the exact solver in a finite time (3600s). Moreover, the framework exhibits strong generalization capabilities by learning meaningful problem structure features. Ablation studies further validate the effectiveness of each module.

*Keywords:* Search and rescue, optimal search problem, learning to optimize, graph neural network

## 1. Introduction

The search process is a key component in numerous humanitarian efforts, including disaster response, mine clearance, and security-related missions, aimed at safeguarding people, assets, and infrastructure from potential or ongoing dangers. Search teams, consisting of human personnel, trained dogs,

aerial vehicles, maritime units, or autonomous robotic systems, are often deployed to locate missing individuals, detect hazardous materials such as land or underwater mines, or identify suspicious activities and unusual behavior (Morin et al., 2023). In the aftermath of natural disasters like earthquakes, floods, and landslides, as well as incidents involving structural collapses or aviation accidents, rapid coordination and execution of Search and Rescue (SAR) missions are essential for locating and retrieving survivors (Kratzke et al., 2010).

Search theory has been steadily progressing since it was first proposed during World War II (Koopman, 1957; Frost, 1999). In recent years it has been formalized as the Optimal Search Problem (OSP), a typical NP-hard optimization task whose solution space grows exponentially with problem scale (Trummel and Weisinger, 1986; Stone et al., 2016). This computational barrier has sparked a surge of research into both exact and heuristic approaches that draw on domain expertise and explicitly exploit problem structure, yielding superior performance on benchmark instances (Stone et al., 2016; Wu et al., 2023). However, these methods are usually tailored to narrowly defined settings. Their effectiveness often degenerates when applied outside the original design scope, where realistic searching conditions can vary dramatically(Raap et al., 2019; Lau et al., 2008). In addition, bilinear constraint linearization preserves convexity but introduces binary variables, increasing complexity (Ma et al., 2022). McCormick relaxations may converge slowly with loose bounds. Balancing linearization fidelity and computational efficiency is critical for performance. While the Markovian structure models stochastic target behavior, the framework lacks mechanisms to address parametric uncertainty in detection probabilities and transition dynamics (Zeng et al., 2025). Hence, further research is needed to overcome these limitations.

Nowadays, researchers have developed a suite of Machine Learning (ML) approaches that learn intrinsic patterns and structures shared by instances from scenarios (Xu et al., 2023). Over the past decades, these methods have generally followed two directions: one line of work embeds neural network modules within classical algorithms, replacing key components such as variable selection, cutting-plane generation, or neighborhood destruction strategies in large neighborhood search (Qu et al., 2022). The other direction, which is the focus of this paper, uses ML to predict high-quality initial feasible solutions that are subsequently refined by the search process. These approaches were originally designed for Mixed-Integer Linear Programming

(MILP) problems, whose constraints are simple and linear, making their structure relatively easy for neural networks to learn (Han et al., 2023; Pu et al., 2024b). However, realistic OSP instances display much greater complexity: they are markedly sparser and governed by intricate, heterogeneous and dynamic constraint sets, which pose significant challenges for learning-based methods. For now, ML has shown remarkable success in solving various MILP problems. Its application to OSP remains largely unexplored, which motivates our work. Here are three key research challenges and objectives derived in summary:

(1) Real-world OSPs involve sparse, dynamic, and highly structured constraints, making them significantly more complex than standard MILPs. A core challenge is developing learning-based methods that effectively capture these intricate patterns while maintaining computational efficiency.

(2) While ML can predict initial solutions, integrating these predictions into a robust search process remains difficult. The study aims to design a hybrid framework where GNNs reduce problem dimensionality and guide a block-structured trust-region search, ensuring fast convergence without sacrificing solution quality.

(3) Existing methods often fail when applied beyond their original design scope. This work seeks to improve adaptability by learning transferable structural patterns, enabling the model to perform well across varying problem scales and dynamic environments (e.g., moving targets or uncertain detection probabilities).

These objectives address critical limitations in traditional OSP solvers while advancing ML-driven optimization for real-world search missions. In light of this, we introduce NeuPath, a new approach designed for realistic OSP scenarios. NeuPath first employs ML to predict an initial solution in a data-driven manner, then uses the supporting knowledge extracted from this prediction to guide a subsequent search toward a final, high-quality solution. Concretely, we linearize the OSP formulation and cast it as a MILP, represent each instance as a bipartite graph, and train a graph neural network (GNN) to approximate the solution distribution across a large amount of instances. The output of GNN serves two purposes: it reduces the effective problem dimension and supplies informative guidance for the search phase.

In addition, we observe that the realistic OSP instances tend to be sparse and exhibit clear block structure among variables, so we reinforce the model with a two-stage aggregation GNN that first aggregates information within each block ("block-in" aggregation) and then aggregates across blocks ("block-out" aggregation). During the search stage, instead of a global trust-region strategy, we adopt a block-based trust-region scheme that fixes variables block by block, which accelerates convergence and yields consistently higher-quality solutions.

Main contributions of this paper are summarized as follows:

(1) We propose NeuPath, a novel data-driven hybrid framework for OSP. It first predicts an initial solution by GNNs and then leverages extracted knowledge to guide a refined search, ensuring high-quality solutions for realistic OSP scenarios.

(2) For the first time, the OSP is conceptualized as a graph combination optimization problem. The OSP is represented in the form of a bipartite graph, which enhances the extraction of problem features and facilitates dimensionality reduction through GNNs.

(3) We enhance the GNN framework with a two-stage aggregation mechanismto improve prediction accuracy and scalability. To address the sparse decision space challenge, we propose a block-wise trust-region scheme that incrementally fixes variables by block.

(4) Experiments demonstrate that NeuPath achieves an average 2.6× speedup over exact solvers (Gurobi, SCIP) in classical OSP scenarios.

The remainder of this paper is structured as follows. Section 2 reviews related work, including both traditional methods and ML approaches. Section 3 formulates the OSP, followed by the details of NeuPath introduced in Section 4. Section 5 presents experimental results for the classic OSP scenario and its variants, benchmarking NeuPath against traditional exact solvers. Section 6 incorporates a comprehensive real-world case study. Finally, Section 7 provides a summary of the theoretical and practical significance of this research, while Section 8 encapsulates the main findings of this paper and outlines directions for future work.

## 2. Related work

Efficiently solving OSPs has significant theoretical and economic implications in operations research and management science, which lead to numerous excellent methods being developed (Pal et al., 2024). In this section, we review problem formulations, traditional algorithms specifically designed for OSP and learning-based optimization methods developed for broader optimization domains while still offering valuable insights applicable to OSP.

### *2.1. OSP formulations*

The OSP has been widely formulated before, with models varying by target motion, sensor capabilities, and searcher constraints. For discrete-time and space problems, Mixed Integer Linear/Nonlinear Programming (MILP/MINLP) is common (Raap et al., 2019). Stewart (1979) first formulated OSP as a MINLP, solved via branch-and-bound (B&B), while Morin et al. (2023) developed MILP models with visibility constraints for commercial exact solvers. Path-constrained searches often use network-based models. Lau et al. (2008) maximized detection probability via longest-path searches in directed acyclic graphs, using dynamic bounds for efficiency. Asfora et al. (2020) expanded this with static bounds and relocation arcs for multi-searcher cases. For kinematics-constrained searches, Sato and Royset (2010) incorporated nonlinear velocity constraints, and Raap et al. (2017) proposed MILP formulations for kinematic feasibility.

Another key framework is the Markov Decision Process (MDP) and Partially Observable Markov Decision Process (POMDP) (Li et al., 2025), which handle uncertainty in target location and sensor data. Eagle (1984) introduced this approach, using Dynamic Programming (DP) for Markovian targets, though scalability remains limited. Cooperative multi-searcher problems employ centralized or decentralized methods (Wang et al., 2025). Negotiation protocols like max-sum algorithms (Fave et al., 2010) balance optimality and communication costs. Hybrid objectives, such as Kullback-Leibler divergence (Furukawa et al., 2007), enhance modeling flexibility. Recent trends favor decomposition methods (e.g., Dantzig-Wolfe) for scalability (Shibasaki et al., 2024), reflecting the trade-off between realism and tractability in OSP modeling.

### *2.2. Traditional optimization algorithms*

As a mature research topic spanning decades, at least three primary strategies have emerged in the OSP field: exact algorithms (Sato and Royset,

2010), approximation algorithms (Griesbach et al., 2025), and heuristic ones (Hong et al., 2009), all extensively surveyed in literature (Raap et al., 2019).

Exact approaches typically exploit problem structure to simplify complex scenarios or adapt classical methodologies like DP and B&B (Morin et al., 2012). For instance, Eagle (1984) proposed a DP formulation treating OSP as a partially observable Markov decision process with Markovian target motion. Similar structural simplifications characterize the B&B methods, first introduced for OSP by Stweart (1980) and improved in the literature (Eagle, 1984), which relax constraints to derive objective function bounds and allow search tree pruning. While provably delivering optimal solutions, exact algorithms become computationally prohibitive for large-scale instances. The approximation algorithms for the OSP provide theoretical guarantees on the quality of the solution relative to the optimal solution (Hermans et al., 2022). These methods typically involve relaxing certain constraints or using simplified problem formulations to obtain solutions in polynomial time. However, even with theoretical guarantees, approximation algorithms may still be too computationally expensive for large-scale problems. Moreover, heuristic methods frequently use domain knowledge or heuristic rules to solve OSP, demonstrating superior practical performance, such as genetic algorithms and ant colony optimization (Berger and Lo, 2015; Morin et al., 2023). While effective for certain problems, these heuristics suffer from limited transferability to similar scenarios and restricted adaptability in rapidly varying environments. For practical-scale problems, they typically lack provable estimates of real performance optimality gaps, raising concerns about their actual relative efficiency (Heng et al., 2024).

### *2.3. Learning-based optimization methods*

In recent years, learning-based optimization methods have demonstrated a remarkable ability to represent the structure of complex problems and capture their intrinsic non-linear patterns in a data-driven manner, thus becoming a highly promising solution paradigm for solving complex optimization problems such as MILP and graph combinatorial optimization (Bengio et al., 2021; Cappart et al., 2023). For example, it has recently emerged as a promising approach to solve combinatorial optimization problems, including the set cover, combinatorial auctions, item placement, workload appointment, et al. (Bengio et al., 2021; Li et al., 2024; Mao et al., 2024). Notably, the OSP is inherently associated with MILP and has frequently been represented as such

in prior research (Raap et al., 2019). Therefore, we first provide a comprehensive summary below, focusing on how machine learning can be applied to improve the efficiency of solving mixed-integer linear programming (MILP) problems. Building on this overview, learning-based optimization approaches for MILP have evolved primarily along two complementary directions.

The first one enhances traditional B&B solvers by integrating neural network components to accelerate key heuristic elements. A primary focus has been on learning branching policies, a direction pioneered by Khalil et al. (2016) who formulated variable selection as a classification problem. Building upon this, Gasse et al. (2019) utilized Graph Convolutional Networks (GCNs) to capture the bipartite graph structure of MILP instances, enabling more effective decisions. Gupta et al. (2020) further advanced this by proposing hybrid models that integrate both local and global information, while more recently, Zarpellon et al. (2021) introduced a parameterized framework for policies that adapt during the search.Recent advancements also include the use of Graph Pointer Networks for variable selection (Wang et al. (2024)) and, notably, a deep symbolic discovery framework that generates explicit, human-readable mathematical rules (Kuang et al. (2024)). Concurrently, Scavuzzo et al. (2024) presented a comprehensive framework focused on the broader, practical integration of ML into branch-and-bound solvers. Attention has also been given to cutting plane selection, where Tang et al. (2020) first framed the problem as a reinforcement learning task to maximize dual bound improvement. Wang et al. (2023) later introduced a hierarchical sequence model to capture temporal dependencies, and Paulus et al. (2022) developed an imitation learning framework to mimic expert policies, achieving superior performance. Beyond these core areas, ML has been applied to other heuristics, such as learning-based node selection by He et al. (2014) and the intelligent scheduling of heuristics by Khalil et al. (2017).

Another major direction leverages ML models to predict full or partial solutions, thereby providing high-quality starting points for traditional solvers Pu et al. (2025). This includes primal heuristic approaches like Neural Diving, where Nair et al. (2021) used graph neural networks to predict and fix variable values—a method later refined with look-ahead mechanisms by Paulus and Krause (2023). As an alternative to direct fixing, Han et al. (2023) proposed the Predict-and-Search (PS) framework to perform a trust-region search around a predicted solution, which Huang et al. (2024) subsequently enhanced with contrastive learning.Subsequently, alternating prediction-correction neural solving framework (Apollo-MILP) advances pre-

diction accuracy via solution correction mechanisms (Liu et al., 2025a). Similarly, ML has been applied to learn policies for Large Neighborhood Search (LNS); Song et al. (2020) developed a general framework for selecting neighborhood structures, while Sonnerat et al. (2021) focused on learning adaptive neighborhood operators specifically for MIPs. Other related contributions include acceleration techniques based on solution prediction. Ding et al. (2020) developed acceleration techniques based on solution prediction, while Khalil et al. (2022) introduced a data-driven framework for guiding combinatorial solvers using graph neural networks.

## 3. Mathematical Formulation

This section presents a comprehensive mathematical formulation of the OSP in discrete time and space. We develop an MILP model to capture the essential characteristics of search operations, including Markovian target motion, exponential detection laws, and operational constraints.

### *3.1. Problem description*

Consider a discrete search environment consisting of $R$ regions where a single searcher must locate a moving target over a finite planning horizon of $T$ time periods. The searcher begins at a predetermined initial location and must visit exactly one region at each time step. The target moves according to a first-order Markovian motion model, and the searcher's objective is to maximize the cumulative probability of target detection throughout the search operation. The search strategy consists of a discrete path planning component and an effort allocation strategy. At each time step, the searcher can move to an adjacent region and perform a search operation with a given detection probability. To promote resource utilization, each region is only allowed to be visited at most once during the entire search operation. The detailed mathematical notations are shown in Table 1.

Table 1: Mathematical notations for classic OSP.

| Symbol/Definition | Description |
|---|---|
| **Sets and indices** | |
| $\mathcal{T} = \{1, 2, \ldots, T\}$ | Set of discrete time periods |
| $\mathcal{R} = \{1, 2, \ldots, R\}$ | Set of spatial regions in the search environment |
| $\mathcal{A}(i) \subseteq \mathcal{R}, \forall i \in \mathcal{R}$ | Set of regions reachable from region $i$ |
| **Parameters** | |
| $\pi_{i1} \geq 0$, $\sum_{i \in \mathcal{R}} \pi_{i1} = 1$ | Initial probability of target presence in region $i$ |
| $P_{ij} \geq 0$, $\sum_{j \in \mathcal{R}} P_{ij} = 1$, $\forall i \in \mathcal{R}$ | Transition probability from region $i$ to region $j$ in the Markovian target motion model |
| $w_t(i, j) \geq 0$, $\forall i, j \in \mathcal{R}$, $\forall t \in \mathcal{T}$ | Detection width parameter for searching region $j$ from position $i$ at time $t$ |
| $r_0 \in \mathcal{R}$ | Predetermined initial searcher position |
| $M > 0$, sufficiently large | Big-M parameter for linearization of bilinear constraints |
| **State variables** | |
| $\pi_{it} \in [0, 1]$, $\forall i \in \mathcal{R}$, $\forall t \in \mathcal{T}$ | Probability of Containment (POC): probability that target is in region $i$ at time $t$ and has not been detected |
| $\mathrm{POD}_t(i, j) = 1 - \exp(-w_t(i, j))$, $\forall i, j \in \mathcal{R}, \forall t \in \mathcal{T}$ | Probability of Detection (POD): conditional probability of a detection when searching region $j$ from position $i$ at time $t$ (exponential detection law) |
| **Decision variables** | |
| $x_{it} \in \{0, 1\}$, $\forall i \in \mathcal{R}$, $\forall t \in \mathcal{T}$ | Binary variable: 1 if searcher visits region $i$ at time $t$, 0 otherwise |
| $q_{it} \in [0, 1]$, $\forall i \in \mathcal{R}$, $\forall t \in \mathcal{T}$ | Probability of Success (POS): Local detection success probability when visiting region $i$ at time $t$ |

### *3.2. Mixed-Integer Nonlinear Programming Formulation*

The complete MINLP formulation for the OSP is presented as follows, where $s_t = \sum_{i \in \mathcal{R}} i \cdot x_{it}$ represents the searcher's position at time $t$.

The objective is to maximize the cumulative probability of detection success, which reflects the expected total opportunities for target detection success during the search operation.

$$\max \quad Z = \sum_{i \in \mathcal{R}} \sum_{t \in \mathcal{T}} q_{it} \cdot x_{it} \tag{1}$$

These initial position constraints establish the searcher's starting position at the predetermined initial region $r_0$. Constraints (2) ensure the searcher begins at region $r_0$, while constraints (3) prevent the searcher from starting at any other region.

$$x_{r_0 1} = 1 \tag{2}$$

$$x_{i1} = 0, \quad \forall i \in \mathcal{R}, i \neq r_0 \tag{3}$$

These movement feasibility constraints enforce the physical movement limitations of the searcher. A region $j$ can only be visited at time $t$ if the searcher was at an adjacent region during the previous time period. The adjacency relationship is defined by the reachability mapping $\mathcal{A}(i)$.

$$x_{jt} \leq \sum_{i \in \mathcal{R}: j \in \mathcal{A}(i)} x_{i(t-1)}, \quad \forall j \in \mathcal{R}, \forall t \in \mathcal{T} \tag{4}$$

Operational constraints (5) ensure that each region is visited at most once during the entire search operation, preventing redundant searches and ensuring efficient resource allocation. Constraint (6) guarantees that the searcher visits exactly one region at each time period, maintaining continuous search operations.

$$\sum_{t \in \mathcal{T}} x_{it} \leq 1, \quad \forall i \in \mathcal{R} \tag{5}$$

$$\sum_{i \in \mathcal{R}} x_{it} = 1, \quad \forall t \in \mathcal{T} \tag{6}$$

The $q_{it}$ represents the searcher's belief about local detection success probability in region $i$ at time $t$. This information state evolves according to Bayesian updating principles, combining prior beliefs, target motion predictions, and search outcomes. These constraints implement the big-M linearization technique (3.3) to handle the bilinear term $\pi_{it} \cdot \text{POD}_t(s_t, i)$.

$$q_{it} = \pi_{it} \cdot \text{POD}_t(s_t, i), \quad \forall i \in \mathcal{R}, \forall t \in \mathcal{T} \tag{7}$$

Constraints (8) initialize the containment probabilities with the prior target distribution. Constraints (9) model the dynamic evolution of containment probabilities, accounting for target motion according to the Markovian transition matrix and reducing probabilities by successful detections. Constraints

(10) ensure that containment probabilities maintain proper normalization, with the sum being less than or equal to 1 due to potential target detection.

$$\pi_{1i} = \pi_{0i}, \quad \forall i \in \mathcal{R} \tag{8}$$

$$\pi_{it} = \sum_{j \in \mathcal{R}} P_{ji} \cdot [\pi_{(t-1)j} - q_{j(t-1)}], \quad \forall i \in \mathcal{R}, \forall t \in \mathcal{T} \tag{9}$$

$$\sum_{i \in \mathcal{R}} \pi_{it} \leq 1, \quad \forall t \in \mathcal{T} \tag{10}$$

*3.3. Linearization of the objective function*

The fundamental challenge in formulating the OSP as a MILP lies in the trilinear term $\pi_{it} \cdot \mathrm{POD}_t(i,j) \cdot x_{it}$ within the objective function (or a major constraint, please adapt based on your model context), which defines the cumulative POS. This term involves the product of two continuous variables ($\pi_{it}$ and $\mathrm{POD}_t(i,j)$) and one binary variable ($x_{it}$), which cannot be directly handled by standard MILP solvers.

To address this nonlinearity, we employ a two-stage linearization strategy. We first linearize the product involving the binary variable and then tackle the remaining bilinear term.

*3.3.1. Stage 1: Linearizing the product with the binary variable*

Our goal is to model a term, let's call it $q_{ijt}$, such that it equals the product $\pi_{it} \cdot \mathrm{POD}_t(i,j)$ if the binary decision variable $x_{it}$ is 1, and 0 otherwise. To achieve this, we first introduce an auxiliary continuous variable $z_{ijt}$ to represent the bilinear part of the product:

$$z_{ijt} = \pi_{it} \cdot \mathrm{POD}_t(i,j) \quad \forall i,j \in \mathcal{R}, \forall t \in \mathcal{T} \tag{11}$$

With $z_{ijt}$ defined, the original trilinear term is equivalent to $z_{ijt} \cdot x_{it}$. We now introduce our target auxiliary variable $q_{ijt}$ and enforce the relationship $q_{ijt} = z_{ijt} \cdot x_{it}$ using a standard big-M formulation (Chen et al., 2011; Zhang and Chen, 2023). The following constraints precisely model this logic:

$$q_{ijt} \leq z_{ijt}, \qquad \forall i,j \in \mathcal{R}, \forall t \in \mathcal{T} \tag{12}$$

$$q_{ijt} \leq M \cdot x_{it}, \qquad \forall i,j \in \mathcal{R}, \forall t \in \mathcal{T} \tag{13}$$

$$q_{ijt} \geq z_{ijt} - M \cdot (1 - x_{it}), \qquad \forall i,j \in \mathcal{R}, \forall t \in \mathcal{T} \tag{14}$$

$$q_{ijt} \geq 0, \qquad \forall i,j \in \mathcal{R}, \forall t \in \mathcal{T} \tag{15}$$

where $M$ is a sufficiently large constant. When $x_{it} = 1$, constraints (12) and (14) combine to enforce $q_{ijt} = z_{ijt}$. Conversely, when $x_{it} = 0$, constraint (13) forces $q_{ijt} \leq 0$, and combined with the non-negativity constraint (15), it ensures $q_{ijt} = 0$. Note that we have corrected the variable index from $q_{it}$ to $q_{ijt}$ to maintain consistency, as its value depends on $j$ through $z_{ijt}$.

It is important to note that for the specific scope of this paper, we assume the Probability of Detection, $\text{POD}_t(i, j)$, to be a known constant parameter rather than a variable. Under this assumption, the term $z_{ijt} = \pi_{it} \cdot \text{POD}_t(i, j)$ is, in fact, already linear with respect to the decision variable $\pi_{it}$. Therefore, the McCormick relaxation described in the subsequent stage is not strictly necessary for solving the current model.

Nevertheless, we present the complete linearization procedure for this bilinear term to establish a more general and forward-looking framework. This detailed methodology is retained deliberately to serve as a direct blueprint for future research extensions where $\text{POD}_t(i, j)$ could be treated as a variable, for instance, depending on dynamic environmental conditions or sensor allocation decisions.

*3.3.2. Stage 2: Linearizing the bilinear term*

The formulation is not yet linear, as the definition of $z_{ijt}$ remains a product of two continuous variables. We address this by replacing the non-convex bilinear term with its convex relaxation using McCormick envelopes (Scott et al., 2011; Mitsos et al., 2009). This technique provides the tightest possible convex relaxation for a bilinear term given the bounds on its variables. Let $[\pi^L, \pi^U]$ and $[\text{POD}^L, \text{POD}^U]$ be the lower and upper bounds for $\pi_{it}$ and $\text{POD}_t(i, j)$, respectively. The bilinear equality $z_{ijt} = \pi_{it} \cdot \text{POD}_t(i, j)$ is replaced by the following four linear inequalities:

$$z_{ijt} \geq \pi^L \cdot \text{POD}_t(i, j) + \text{POD}^L \cdot \pi_{it} - \pi^L \cdot \text{POD}^L \tag{16}$$

$$z_{ijt} \geq \pi^U \cdot \text{POD}_t(i, j) + \text{POD}^U \cdot \pi_{it} - \pi^U \cdot \text{POD}^U \tag{17}$$

$$z_{ijt} \leq \pi^U \cdot \text{POD}_t(i, j) + \text{POD}^L \cdot \pi_{it} - \pi^U \cdot \text{POD}^L \tag{18}$$

$$z_{ijt} \leq \pi^L \cdot \text{POD}_t(i, j) + \text{POD}^U \cdot \pi_{it} - \pi^L \cdot \text{POD}^U \tag{19}$$

*3.3.3. Determining tight bounds*

The effectiveness of both the big-M method and McCormick relaxation critically depends on the tightness of the variable bounds and the constant $M$. Loose bounds can lead to a weak LP relaxation and significantly deteriorate

the solver's performance. We therefore establish the tightest possible bounds as follows:

$$\pi^L = 0, \quad \pi^U = 1 \tag{20}$$

$$\mathrm{POD}^L = 0, \quad \mathrm{POD}^U = 1 - \exp(-w_t(i,j)^{\max}) \tag{21}$$

The bounds for $\pi_{it}$ are naturally [0, 1] as it represents a probability. The bounds for $\mathrm{POD}_t(i,j)$ are also non-negative, with an upper bound derived from the maximum possible value of its parameter, $w_t(i,j)^{\max} = \max_{i,j,t} w_t(i,j)$.

Finally, to ensure the strength of the big-M formulation, the constant $M$ should be set to the tightest possible upper bound of the variable $z_{ijt}$. This upper bound is simply the product of the upper bounds of $\pi_{it}$ and $\mathrm{POD}_t(i,j)$:

$$M = \pi^U \cdot \mathrm{POD}^U = 1 \cdot (1 - \exp(-w_t(i,j)^{\max})) = 1 - \exp(-w_t(i,j)^{\max}) \tag{22}$$

By replacing the original trilinear term with the auxiliary variable $q_{ijt}$ and adding constraints (12)-(19) along with the specified bounds, we obtain a valid linear representation of the problem.

## 4. Method

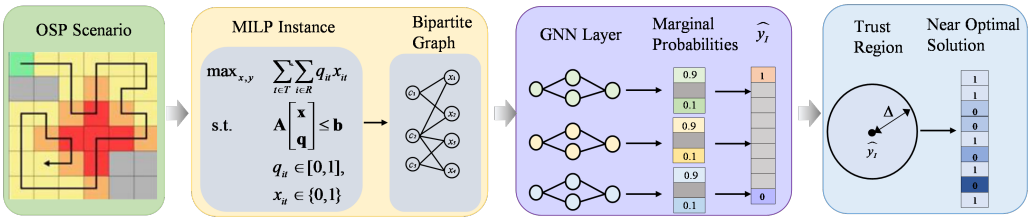


Figure 1 An overview of the proposed NeuPath framework, illustrating its two main stages: GNN-based learning for probability prediction and trust-region optimization for solution search.

We propose the NeuPath that integrates learning (Section 4.2) with optimization (Section 4.3), where the complete framework with two steps is shown in Fig. 1. For the learning step, the NeuPath framework transforms OSP combinatorial optimization problems into MILP instances, representing them as bipartite graphs with variable and constraint nodes. Then, we employ a multi-layer GNN with specialized aggregation mechanisms (detailed

in Fig. 2) to predict marginal probabilities for each variable. For the optimization step, instead of directly fixing variables, the predicted probabilities guide the construction of a trust region where a near-optimal feasible solution is searched within the intersection of the original feasible region and the defined neighborhood ball.

### *4.1. Notation Summary*

For clarity, the key mathematical notations used throughout the methodology are summarized in Tables 2 and 3. Table 2 covers notations related to the graph representation and the GNN architecture, while Table 3 details symbols pertaining to the learning and optimization components.

Table 2: Notations for graph representation and GNN architecture.

| Symbol | Description |
|---|---|
| $\mathcal{G}$ | The bipartite graph representing the MILP instance. |
| $\mathcal{V}, \mathcal{C}, \mathcal{E}$ | Sets of variable nodes, constraint nodes, and edges in the bipartite graph $\mathcal{G}$, respectively. |
| $\mathcal{V}_x, \mathcal{V}_q$ | Subsets of $\mathcal{V}$ representing binary visiting variables and continuous probability variables, respectively. |
| $\mathbf{f}_v, \mathbf{f}_c, \mathbf{f}_{vc}$ | Feature vectors for a variable node $v \in \mathcal{V}$, a constraint node $c \in \mathcal{C}$, and an edge $(v, c) \in \mathcal{E}$. |
| $n, m$ | Number of variable nodes ($\lvert\mathcal{V}\rvert$) and constraint nodes ($\lvert\mathcal{C}\rvert$), respectively. |
| $\mathbf{h}_i^{(l)}$ | Hidden state (embedding) of node $i$ at the $l$-th layer of the GNN. |
| $\mathcal{N}(i)$ | The set of neighboring nodes of node $i$ in the graph. |
| $\text{MLP}_{\text{c}}^{(l)}, \text{MLP}_{\text{v}}^{(l)}$ | Multi-Layer Perceptrons for constraint and variable node updates at the $l$-th layer, respectively. |
| $\mathcal{B}$ | The set of blocks in the block-structured aggregation. |
| $B, B_k$ | A block of nodes, often the $k$-th block in the partitioned set $\mathcal{B}$. |
| $\beta_{kB}^{(l)}$ | Attention weight between node $k$ and block $B$ at the $l$-th layer. |
| $U^{(l)}, W^{(l)}$ | Learnable weight matrices at the $l$-th layer. |
| $\text{MLP}_{\text{b}}^{(l)}$ | Multi-Layer Perceptron for block-level aggregation at the $l$-th layer. |

Table 3: Notations for training and optimization.

| Symbol | Description |
|---|---|
| $x_{it}^*$ | Ground truth binary label for the variable $x_{it}$ (typically from an expert or optimal solution). |
| $\hat{x}_{it}$ | Predicted probability for the variable $x_{it}$ being 1, output by the GNN. |
| $p_\theta$ | The vector of predicted probabilities for all variables, $p_\theta = [\hat{x}_{1t}, \hat{x}_{2t}, ..., \hat{x}_{nt}]$. |
| $\mathcal{L}_{CE}$ | Standard cross-entropy loss function. |
| $\mathcal{L}_{FL}$ | Focal loss, a modified cross-entropy loss to address class imbalance. |
| $\gamma$ | Focusing parameter in the focal loss ($\gamma \geq 0$). |
| $I_0, I_1$ | Index sets for variables with the smallest ($I_0$) and largest ($I_1$) predicted probabilities. |
| $k_0, k_1$ | Number of variables to select for sets $I_0$ and $I_1$, respectively. |
| $\delta_d$ | Binary slack variable indicating if the assignment for variable $d$ is deviated from its initial fixed value. |
| $\Delta$ | Global trust region budget; the maximum number of allowed deviations across all variables. |
| $\mathcal{I}_{0,k}, \mathcal{I}_{1,k}$ | Index sets for the bottom ($\mathcal{I}_{0,k}$) and top ($\mathcal{I}_{1,k}$) predicted probability variables within block $B_k$. |
| $n_{0,k}, n_{1,k}$ | Number of variables to select for sets $\mathcal{I}_{0,k}$ and $\mathcal{I}_{1,k}$ within each block $B_k$, respectively. |
| $\mathcal{D}$ | The union of all indices selected for soft fixing across all blocks, $\mathcal{D} = (\bigcup_k \mathcal{I}_{1,k}) \cup (\bigcup_k \mathcal{I}_{0,k})$. |

### *4.2. Learning step*

#### *4.2.1. Problem representation*

The OSP's MILP formulation is represented as a bipartite graph $\mathcal{G} = (\mathcal{V}, \mathcal{C}, \mathcal{E})$, where $\mathcal{V}$ represents the variable nodes, $\mathcal{C}$ represents the constraint nodes, and $\mathcal{E}$ denotes the edges connecting variables to constraints (Gasse et al., 2019; Han et al., 2023; Pu et al., 2024a).

Variable nodes $\mathcal{V} = \mathcal{V}_x \cup \mathcal{V}_q$ represent the matrices of decision variables in our OSP formulation, where $\mathcal{V}_x$ represents the matrix of binary visiting variables $x_{it}$ and $\mathcal{V}_q$ represents the matrix of continuous probability variables $q_{it}$. Each variable node $v \in \mathcal{V}$ is associated with a feature vector $\mathbf{f}_v$ encoding variable-specific information such as variable type (binary/continuous), bounds, current LP solution values, and reduced costs.

Constraint nodes $\mathcal{C}$ denote the matrices of various constraint types in the OSP model, including detection probability constraints, connectivity constraints, node visiting limits, and temporal constraints. Each constraint node $c \in \mathcal{C}$ is characterized by a feature vector $\mathbf{f}_c$ capturing constraint properties such as constraint type, tightness in the current LP relaxation, dual solution values, and constraint coefficient statistics.

Edges $(v, c) \in \mathcal{E}$ exist between a variable node $v$ and a constraint node $c$ if variable $v$ appears in constraint $c$ with a nonzero coefficient. Each edge is associated with a feature vector $\mathbf{f}_{vc}$ encoding the constraint coefficient value and its normalized form.

This bipartite graph representation naturally captures the structure of the OSP while remaining invariant to variable and constraint ordering (Liu et al., 2025b). The GNN can leverage this graph structure to learn meaningful embeddings that encode both local variable-constraint relationships and global problem characteristics, enabling effective variable selection policies for B&B optimization.

*4.2.2. GNN architecture*

Our objective is to learn an embedding vector for each variable that simultaneously captures variable-variable interactions and variable-constraint relationships. Previous approaches typically employed two graph convolution layers: one aggregating information from variables to constraints, and the other aggregating information from constraints to variables. However, we observe that MILP instances derived from OSP scenarios exhibit a strong sparse and block structure. This structure arises because OSP scenarios involve selecting only a specific region to search at the current time. Unfortunately, this sparse and block structure presents a significant challenge for GNNs, indicating the need for a GNN with strong expressive power. One intuitive solution is to increase the number of GNN layers, but this will lead to the problem of over-smoothing.

To address this challenge, we design a Block-Aggregation Neural Network (BAN) to effectively capture the block structure of OSPs. The network comprises two main components: block-in aggregation and block-out aggregation, as detailed in Fig. 2. The left panel illustrates the block-in aggregation where BAN Layer A processes variable nodes (yellow circles) and constraint nodes (pink circles) to generate intermediate node features. The right panel shows the block-out aggregation where BAN Layer B further refines these features through cross-type node interactions. The dashed circles highlight the

neighborhood aggregation scope, with orange arrows indicating the direction of information flow between different node types in the bipartite structure.

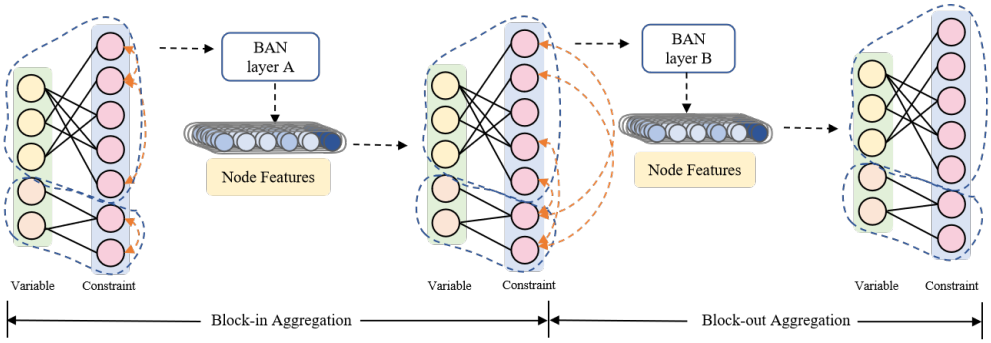


Figure 2 Detailed architecture of the GNN layers. It shows the block-in and block-out aggregation mechanisms for bipartite graph processing.

In the block-in aggregation phase, we perform two similar layers of graph convolution operations within each block:

$$\mathbf{h}_i^{(l)} = \mathrm{MLP}_{\mathrm{c}}^{(l)} \left( \mathbf{h}_i^{(l-1)}, \sum_{u \in \mathcal{N}(i)} \mathbf{h}_u^{(l-1)} \right) i \in \{n+1, \ldots, n+m\}, \tag{24}$$

$$\mathbf{h}_j^{(l)} = \mathrm{MLP}_{\mathrm{v}}^{(l)} \left( \mathbf{h}_j^{(l-1)}, \sum_{u \in \mathcal{N}(j)} \mathbf{h}_u^{(l-1)} \right) j \in \{1, \ldots, n\}, \tag{25}$$

where $\mathcal{N}(i)$ denotes neighbors of node $i$, $n$ and $m$ are variable and constraint counts respectively, with $\mathrm{MLP}_{\mathrm{c}}^{(l)}$ and $\mathrm{MLP}_{\mathrm{v}}^{(l)}$ being two-layer perceptrons with ReLU activations.

For the block-out aggregation, we perform an extra aggregation operation to strengthen the relationship between different blocks:

$$\mathbf{h}_k^{(l+1)} = \mathrm{MLP}_{\mathrm{b}}^{(l)} \left( W^{(l)} \mathbf{h}_k^{(l)} + \sum_{B \in \mathcal{B}} \beta_{kB}^{(l)} \left( U^{(l)} \sum_{j \in B} \mathbf{h}_j^{(l)} \right) \right) \tag{26}$$

where $\mathcal{B}$ is the block set, $\beta_{kB}^{(l)}$ is the attention weight between node $k$ and block $B$, $U^{(l)}, W^{(l)}$ are learnable weights, and $\mathrm{MLP}_{\mathrm{b}}^{(l)}$ being two-layer perceptrons with ReLU activations.

*4.2.3. Model training*

To train the GNN model, we initially consider the standard cross-entropy loss function:

$$\mathcal{L}_{CE} = -\frac{1}{n}\sum_{i=1}^{n}[x_{it}^{*}\log(\hat{x}_{it}) + (1 - x_{it}^{*})\log(1 - \hat{x}_{it})], \tag{27}$$

where $x_{it}^{*}$ is the ground truth label for variable $x_{it}$, $\hat{x}_{it}$ is the predicted probability, and $n$ is the total number of variables.

However, the OSP exhibits a significant class imbalance, with most variables in the optimal solution being 0 and only a few being 1. This imbalance makes it difficult for the model to learn to predict the minority class accurately using standard cross-entropy loss.

Therefore, to address the class imbalance issue, we introduce focal loss, which is a modified version of cross-entropy loss that assigns higher weights to hard-to-classify examples:

$$\mathcal{L}_{FL} = -\frac{1}{n}\sum_{i=1}^{n}[(1 - \hat{x}_{it})^{\gamma}x_{it}^{*}\log(\hat{x}_{it}) + \hat{x}_{it}^{\gamma}(1 - x_{it}^{*})\log(1 - \hat{x}_{it})], \tag{28}$$

where $\gamma > 0$ is a focusing parameter that adjusts the weight assigned to hard-to-classify examples. A higher value of $\gamma$ gives more weight to examples with smaller predicted probabilities for the correct class. The focal loss helps our model focus on the minority class and the difficult examples, improving its ability to predict the decision boundary accurately. This is particularly important for the OSP, where accurately predicting the rare 1-valued variables is crucial for finding good solutions.

*4.3. Optimization step*

In this section, we describe how to leverage the GNN predictions to guide the optimization process. Our approach follows a predict-and-search paradigm Han et al. (2023): first obtaining an initial solution from the GNN model, and then refining it with trust-region search techniques. We present both the basic trust-region method and its block-structured extension.

*4.3.1. Trust-region search*

Once trained, the GNN model predicts probabilities $p_\theta \in [0, 1]^n$ for each variable in the optimal solution of the OSP instance. We employ a trust-

region search by first sorting variables in ascending order of predicted probability. Define $I_0$ as the indices of the $k_0$ variables with the smallest probabilities and $I_1$ as the indices of the $k_1$ variables with the largest probabilities. For each $d \in I_0 \cup I_1$, create a binary slack variable $\delta_d$ and enforce:

$$x_d \leq \delta_d, \quad \text{for } d \in I_0 \tag{29}$$

$$x_d \geq 1 - \delta_d, \quad \text{for } d \in I_1 \tag{30}$$

The trust region constraints are implemented as $\sum_{d \in I_0 \cup I_1} \delta_d \leq \Delta$, where $\Delta$ bounds the number of permitted modifications to the initially fixed variables. The modified MILP instance with these constraints and slack variables. This formulation allows modifying at most $\Delta$ originally fixed assignments from $I_0 \cup I_1$, with $\delta_d = 1$ indicating a reversed assignment.

*4.3.2. Block-structured trust-region search*

Although the global trust region effectively restricts the search space, MILP problems often exhibit structures where variables can be grouped based on shared properties Guo et al. (2024). For instance, variables with similar GNN-predicted probabilities can be considered a group, suggesting they might behave similarly in the optimal solution. We exploit this by introducing a block-structured trust-region search.

To implement this, we partition the variables based on their GNN-predicted probabilities. First, all binary variables are sorted globally according to their probability scores. This sorted list is then partitioned into $K$ contiguous blocks $\{B_1, B_2, \ldots, B_K\}$ of a predefined, fixed size. This strategy ensures that variables within the same block exhibit similar probability scores, allowing for targeted control.

To refine the search, within each block $B_k$, we identify a subset of variables for soft fixing based on their local ranking. Specifically, we select the top $n_{1,k}$ variables with the highest probabilities (candidates for being 1) and the bottom $n_{0,k}$ variables with the lowest probabilities (candidates for being 0). Let $\mathcal{I}_{1,k}$ be the indices of the top variables in block $k$, and $\mathcal{I}_{0,k}$ be the indices of the bottom variables.

The trust-region constraints are then applied to these selected variables across all blocks. We introduce a binary slack variable $\delta_d$ for each selected variable $d$. The block-aware trust region system becomes:

$$x_j \leq \delta_j, \quad \forall j \in \bigcup_k \mathcal{I}_{1,k} \tag{31}$$

$$x_i \geq 1 - \delta_i, \quad \forall i \in \bigcup_k \mathcal{I}_{0,k} \tag{32}$$

$$\sum_{d \in \mathcal{D}} \delta_d \leq \Delta, \quad \mathcal{D} = \left( \bigcup_k \mathcal{I}_{1,k} \right) \cup \left( \bigcup_k \mathcal{I}_{0,k} \right) \tag{33}$$

Here, $\delta_d = 1$ indicates that the final assignment of variable $d$ deviates from its GNN-guided initial assignment. The global trust-region constraint, Equation (33), limits the total number of such deviations across all blocks to a budget $\Delta$. This block-aware extension offers more fine-grained control by focusing the search on variables grouped by prediction confidence, while still leveraging a global budget to maintain solver tractability.

### *4.4. Complexity analysis*

The complete framework is summarized in Algorithm 1, integrating GNN-based initialization with block-structured trust-region optimization to efficiently guide the MILP solver. At the learning step, the algorithm constructs a bipartite graph representation $G$ of the MILP instance and partitions variables into $K$ blocks using "InitStructuralBlocks", which leverages problem structure (e.g., temporal stages). For each block $B_k$, "BlockInAggregate" performs localized message passing to capture intra-block dependencies, yielding block representations $H_k$. These are combined via "BlockOutAggregate" to model inter-block relationships. The trained GNN $F_\theta$ then produces probability predictions $p_\theta \in [0,1]^n$ indicating each variable's likelihood of being 1 in the optimal solution. The optimization step uses these predictions to construct trust regions by selecting high-confidence variables within each block and introducing appropriate constraints through slack variables $\delta_d$.

Complexity analysis reveals that the learning step requires a single GNN forward pass with linear complexity $\mathcal{O}(|\mathcal{V}|+|\mathcal{C}|+|\mathcal{E}|)$. The optimization step involves solving a MILP with exponential worst-case complexity $\mathcal{O}(2^{|\mathcal{V}_{\text{bin}}|})$, though our trust-region heuristic significantly prunes the search space. Constraint generation is efficient at $\mathcal{O}(|\mathcal{V}| \log |\mathcal{V}|)$ due to block-wise processing.

This hybrid approach leverages inexpensive neural predictions to accelerate the expensive combinatorial optimization, achieving practical runtime improvements while maintaining optimality guaranties through the exact solving of the constrained MILP.

**Algorithm 1:** NeuPath Algorithm

**Input:** OSP MILP instance $\mathcal{M}$; trained GNN $F_\theta$; number of blocks $K$; trust-region radius $\Delta$; thresholds $k_0, k_1$

**Output:** Feasible solution $x^*$

**Learning step: GNN prediction via in-block aggregation**
$G \leftarrow$ `BuildBipartiteGraph`$(\mathcal{M})$;
$\{B_1, \ldots, B_K\} \leftarrow$ `InitStructuralBlocks`$(G, K)$;
**for** $k = 1$ **to** $K$ **do**
  $H_k \leftarrow$ `BlockInAggregate`$(G, B_k)$
**end**
$H \leftarrow$ `BlockOutAggregate`$(\{H_k\}_{k=1}^{K})$ $p_\theta \leftarrow F_\theta(H)$
**Optimization step: Trust-region construction**
$D \leftarrow \emptyset$;
**for** $k = 1$ **to** $K$ **do**
  $I_{1,k} \leftarrow$ Top-$k_1$ by $p_\theta$ in $B_k$;
  $I_{0,k} \leftarrow$ Bottom-$k_0$ by $p_\theta$ in $B_k$;
  **foreach** $d \in I_{1,k} \cup I_{0,k}$ **do**
    create binary slack $\delta_d$;
    **if** $d \in I_{0,k}$ **then**
      add constraint $x_d \leq \delta_d$;
    **else**
      add constraint $1 - x_d \leq \delta_d$;
    **end**
    $D \leftarrow D \cup \{d\}$;
  **end**
**end**
add constraint $\sum_{d \in D} \delta_d \leq \Delta$;
$\mathcal{M}' \leftarrow \mathcal{M}$ with new slack variables and constraints;
$x^* \leftarrow$ `Solve`$(\mathcal{M}')$;
**return** $x^*$;

## 5. Experiments

### *5.1. Experiments settings*

#### *5.1.1. Configurations for OSP formulation*

There is no publicly available standard dataset for OSP, we conducted mathematical modeling and random initialization to the greatest extent possible in accordance with the requirements of actual search scenarios. Focusing on land search and rescue as well as maritime search and rescue as our case backgrounds, we assume that the targets' motion is either static or random, while temporarily disregarding sensor uncertainties. We evaluate NeuPath on two distinct experimental scenarios to comprehensively demonstrate its performance.

**Static Scenario (Ai et al., 2021):** In this setting, we assume the initial target existence probability remains constant throughout the optimization process. This scenario represents traditional OSP instances where the search environment and target locations do not change during the solving process. The static nature allows us to focus on the fundamental optimization capabilities of different approaches without the additional complexity of environmental dynamics.

**Random Scenario (Morin et al., 2023):** In this more challenging setting, we assume that the target positions change randomly during the subsequent search process. This dynamic scenario simulates real-world search applications where target locations may evolve over time, requiring adaptive optimization strategies. The random changes introduce uncertainty and test the robustness of optimization methods under non-stationary conditions.

#### *5.1.2. Datasets description*

The experimental spatial grids ranged from $9 \times 9$ to $12 \times 12$ cells, corresponding to search areas of approximately 81 to 144 $\mathrm{NM}^2$ (square nautical miles), under the assumption that each grid cell represents a $1 \times 1$ nautical mile segment. This spatial resolution aligns with typical coastal SAR missions conducted by coast guard units. For large-scale oceanic SAR operations, each cell can represent broader geographical units (e.g., $10 \times 10$ nm), allowing the proposed approach to be scaled to varying operational domains.

Temporal parameters were configured to 30–50 discrete time steps, with each step representing a 10 minute operational interval. This setup yields mission durations of approximately 5–8 hours, which is consistent with real-

world SAR operational windows prior to resource redeployment or reassessment of mission strategy.

*5.1.3. Datasets generation*

To rigorously evaluate our approach, we generated a comprehensive dataset of OSP instances designed to exhibit diverse and challenging search scenarios. The entire generation process is deterministic, governed by a fixed random seed, ensuring full reproducibility of our experimental results. The dataset is partitioned into a training set and a test set, whose parameters are detailed below.

The problem definition for each instance encompasses both the target's probability distribution and the searcher's operational constraints. Let $\mathcal{G}$ be the set of all cell coordinates in the grid. The initial POC distribution is synthesized using a mixture model of three two-dimensional multivariate Gaussian distributions. The unnormalized POC, denoted $P_0'(\mathbf{x})$ for a cell at coordinate $\mathbf{x} \in \mathcal{G}$, is computed as the arithmetic mean of three Gaussian Probability Density Functions (PDFs):

$$P_0'(\mathbf{x}) = \frac{1}{3}\sum_{k=1}^{3}\mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k), \tag{23}$$

where $\mathcal{N}(\mathbf{x}|\mu_k, \Sigma_k)$ is the PDF of a multivariate Gaussian. For $k \in \{1, 2, 3\}$, the mean vectors $\mu_k$ are sampled uniformly from the continuous space $[0, G)\times [0, G)$, where $G$ is the number of grids. The covariance matrices $\Sigma_k$ are deterministically defined as a function of $G$:

$$\Sigma_1 = \begin{bmatrix} G/3 & 0.14G \\ 0.14G & G/2 \end{bmatrix}, \quad \Sigma_2 = \begin{bmatrix} G/3 & 0.16G \\ 0.16G & G/2 \end{bmatrix}, \quad \Sigma_3 = \begin{bmatrix} G/3 & 0.20G \\ 0.20G & G/2 \end{bmatrix}.$$

This distribution is subsequently normalized to yield the final POC, $P_0(\mathbf{x})$, ensuring it forms a valid probability mass function over the grid:

$$P_0(\mathbf{x}) = \frac{P_0'(\mathbf{x})}{\sum_{\mathbf{z}\in\mathcal{G}} P_0'(\mathbf{z})}. \tag{24}$$

The search itself is constrained by the agent's dynamics. The search commences at the first node (top-left corner) at time step $t = 1$. At each subsequent step, the agent can transition to one of the four cardinally adjacent nodes. The search path must adhere to the rules that exactly one node is visited at each time step and no node is visited more than once.

Table 4: MILP Instance Statistical Information.

| Size | Constrs. | Vars. | Binary Var. | Cont. | Density |
|---|---|---|---|---|---|
| (9, 30) | 9831 | 4860 | 2430 | 2430 | 0.0006 |
| (10, 30) | 12130 | 6000 | 3000 | 3000 | 0.0005 |
| (10, 40) | 16140 | 8000 | 4000 | 4000 | 0.0004 |
| (11, 40) | 19521 | 9680 | 4840 | 4840 | 0.0003 |
| (11, 50) | 24371 | 12100 | 6050 | 6050 | 0.0002 |
| (12, 50) | 28994 | 14400 | 7200 | 7200 | 0.0002 |

This generation methodology produces a diverse and reproducible benchmark dataset. Table 4 summarizes the dimensions of the generated MILP insances across these configurations. The numbers of constraints, variables, binary variables, continuous variables and density are presented. To obtain the labels (denoted as Best-known Solution) that are used to supervise the model, we solve each generated instance using Gurobi with extended time limits (3600 seconds). These solutions serve as benchmarks for evaluating the performance of different optimization methods. We generate 400 instances for training and 50 instances for testing. And for the training stage, we use 80% of the total instances to train and the remaining for validation.

*5.1.4. Metrics*

We evaluate our approach against two state-of-the-art exact solvers: Gurobi (commercial MIP solver) and SCIP (open-source alternative), both configured with default settings and equal computational resources. The OSP instances are formulated as MILPs to benchmark solution quality and efficiency.

For relatively simple static instances, we measure the time required to reach the optimal solution. This metric directly assesses the computational efficiency of different approaches when the search environment remains constant. We record both the time to first feasible solution and the time to proven optimality, providing insights into the convergence behavior of different methods.

For more complex random scenarios, we evaluate the gap between objective function values achieved by different methods within the same time limit (360 seconds). The gap is calculated relative to the best-known solution (BKS) obtained by running Gurobi for 3600 seconds (Liu et al., 2025a). This

metric is defined as:

$$\text{Gap} = \frac{|\text{NeuPath\_Obj} - \text{BKS}|}{|\text{BKS}|} \times 100\% \tag{34}$$

Additionally, we track the convergence behavior over time and report the number of instances solved to optimality within the given time limits. We also measure solution robustness by evaluating performance variance across multiple runs with different random seeds.

*5.1.5. Implementations*

All experiments are conducted on a single machine equipped with NVIDIA GeForce GPUs (24GB VRAM each, RTX 3090) and Intel(R) Xeon(R) E5-2667 v4 CPUs @ 3.20GHz. This hardware configuration ensures sufficient computational resources for both our approach and the baseline solvers.

We implement our optimization framework using Python 3.8 with NumPy and SciPy for mathematical computations. The baseline solvers (Gurobi 9.5 and SCIP 8.0) are accessed through their respective Python APIs to ensure consistent interface and result collection. All problem instances are generated using standardized random number generators to ensure reproducibility.

*5.2. Training details*

Our experimental hyperparameters are consistent with the predict-and-search methodology. However, we introduce several key modifications in our block trust region search approach. Specifically, we employ a proportional variable fixing strategy where the first 80% of variables are fixed to 0, while the remaining variables are not constrained to 1. The trust region parameter $\delta$ is set to 10% of the total number of fixed variables. For the focal loss function, we configure $\alpha = 0.9$ and $\gamma = 2.0$ to address class imbalance issues effectively. Fig. 3 shows that the validation loss curves demonstrate several important characteristics.

All instances exhibit a sharp decrease in validation loss during the initial training epochs (0-200), indicating that the model quickly learns the fundamental patterns in the optimization landscape. This rapid convergence suggests that our network architecture and training methodology are well-suited for the problem domain.

After the initial rapid descent, the validation loss stabilizes across all problem instances, typically converging within 500-800 epochs. The converged

loss values vary according to problem complexity, with larger instances generally achieving higher final loss values, which is expected given the increased problem difficulty.

The convergence patterns remain remarkably consistent across different problem scales. Instance $(9, 30)$ converges to approximately 22.0, while larger instances such as $(12, 50)$ stabilize around 45-50. This scalability demonstrates the robustness of our approach across varying problem dimensions.

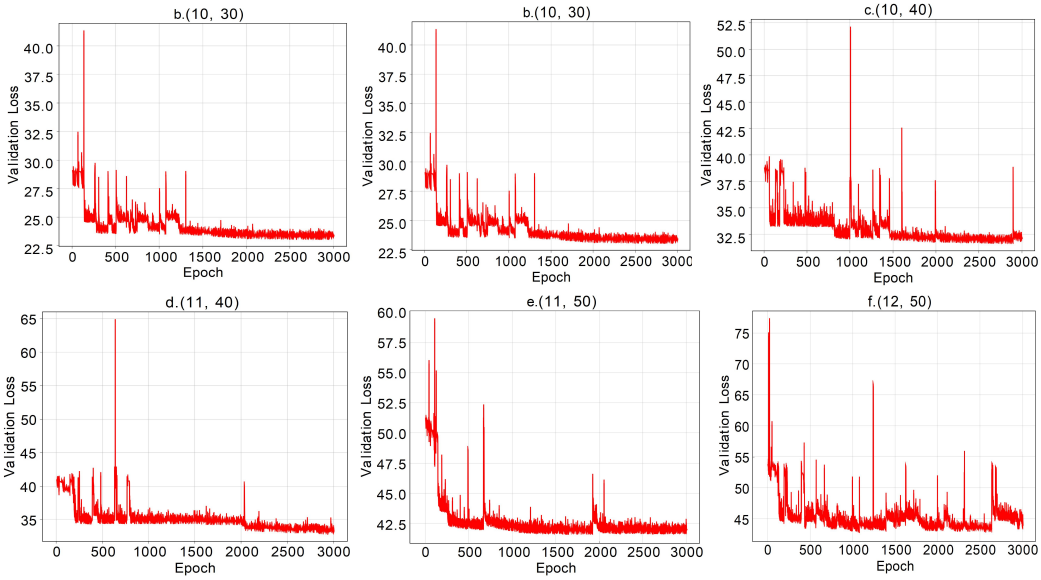


Figure 3 Training curve. It illustrates the training convergence behavior of our neural network model across six different problem instances of varying scales, ranging from $(9, 30)$ to $(12, 50)$.

While occasional spikes appear in some training curves (notably in instances $(11, 40)$ and $(12, 50)$), these fluctuations are minimal and do not compromise overall convergence stability. The absence of significant oscillations or divergent behavior indicates that our learning rate scheduling and regularization strategies are appropriately configured.

The smooth convergence of validation loss without significant overfitting patterns suggests that our model maintains good generalization capability across different problem instances. This is particularly important for optimization problems where the model must perform well on unseen instances.

The experimental results demonstrate that our neural network model achieves excellent convergence properties, successfully learning the complex

mapping between problem instances and high-quality solutions across multiple problem scales. The consistent convergence behavior validates the effectiveness of our proposed methodology and hyperparameter configuration.

### *5.3. Results on the static scenario*

Our approach demonstrates consistent and substantial performance improvements over both professional solvers across all tested problem instances. Table 5 presents the solving times for the static scenario, where our method achieves significant speedups regardless of the underlying solver.

Table 5: Comparison of solving performance between our approach and baseline methods for a static scenario. We build the NeuPath on Gurobi and SCIP, respectively. 80% variables fixed to 0, trust region $\delta = 10\%$ of fixed variables, and focal loss with $\alpha = 0.9$, $\gamma = 2.0$. Time to optimal solution reported. **Best values** in bold show improvement over baseline solvers.

| $(g, T)$ | (9, 30) | (10, 30) | (10, 40) | (11, 40) | (11, 50) | (12, 50) |
|---|---|---|---|---|---|---|
| Gurobi | 0.46 | 0.60 | 26.99 | 30.47 | 55.50 | 210.19 |
| Ours+Gurobi | **0.14** | **0.15** | **15.17** | **22.74** | **27.56** | **84.76** |
| Improvement | 3.29 × | 4.00 × | 1.78 × | 1.34 × | 2.01 × | 2.48 × |
| SCIP | 234.26 | 386.09 | 592.52 | 697.01 | 796.23 | 1286.80 |
| Ours+SCIP | **58.76** | **139.05** | **256.14** | **304.80** | **326.75** | **489.72** |
| Improvement | 3.99 × | 2.78 × | 2.31 × | 2.29 × | 2.44 × | 2.63 × |

When integrated with Gurobi, our approach shows remarkable efficiency gains across all problem sizes. The most impressive improvements occur on smaller instances, with the (10,30) configuration achieving a 4.00× speedup (75.0% time reduction). For medium-sized problems, we observe speedups ranging from 1.34× to 2.01×, while the largest tested instance (12,50) still achieves a notable 2.48× improvement, reducing solving time from 210.19 seconds to 84.76 seconds.

The integration with SCIP yields even more pronounced absolute time savings due to SCIP's longer baseline solving times. Our method achieves speedups ranging from 2.29× to 3.99×, with the (9,30) instance showing the highest improvement of 3.99× (74.9% time reduction). Notably, the absolute time savings are substantial—for the largest instance (12,50), our approach reduces solving time from over 21 minutes to approximately 8 minutes.

The results reveal interesting scalability patterns. Smaller instances show the higher improvements (up to 4.00× with Gurobi), and our method still

maintains consistent performance advantages as problem complexity increases. The improvement ratios remain over 2× for most instances with SCIP and over 1.3× with Gurobi, demonstrating the robustness of our approach across different problem scales.

The consistent improvements across both Gurobi and SCIP highlight the general applicability of our method. This solver-agnostic performance enhancement suggests that our approach successfully addresses fundamental computational bottlenecks in OSP, rather than exploiting solver-specific optimizations.

### *5.4. Results on the random scenario*

We also evaluate our approach on a more challenging random scenario, where the target's movement is random. Fig. 4 presents the experiment results within 360s for this scenario. The results for the random scenario demonstrate that our approach maintains competitive performance compared to professional solvers, suggesting that our method exhibits robustness to different target movement patterns. To further evaluate the solution quality, we analyze the primal gap, which measures the difference between the objective value of a solution and the best known objective value, as shown in Fig. 4.

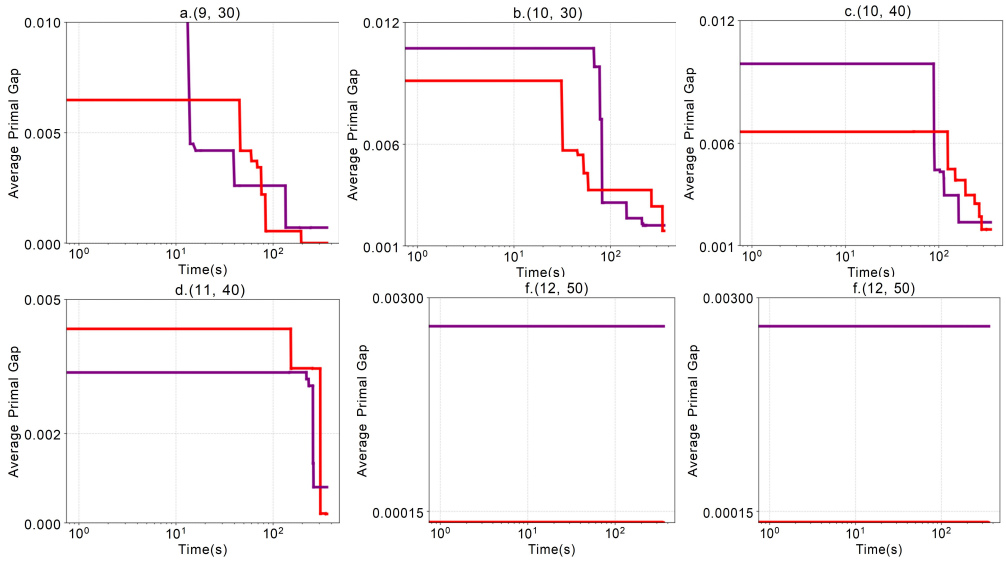


Figure 4 The primal gap evolution over time for different methods across various scale instances. The red line denotes the convergence of our proposed method, while the purple line illustrates the convergence achieved by the Gurobi solver.

The results reveal interesting performance characteristics across different problem sizes. For smaller instances such as (9,30) and (10,30), both methods achieve comparable convergence rates, with each approach demonstrating advantages at different time intervals. As problem complexity increases to medium-scale instances like (10,40) and (11,40), the performance gap becomes more pronounced, with our approach generally achieving faster convergence to lower primal gaps.

Notably, for the larger instances (11,50) and (12,50), our method demonstrates superior performance, maintaining consistently lower primal gaps throughout the optimization process. The (12,50) instance particularly highlights our approach's scalability advantages, where we achieve and maintain near-optimal solutions significantly faster than the baseline methods.

### *5.5. Results on more comparative experiments*

To comprehensively evaluate the performance of NeuPath in static scenarios, we conducted a comparative study with two sets of experiments on instances with parameters (11, 40) and (12, 50). The benchmark algorithms include a classical heuristic, Ant Colony Optimization (ACO) (Morin et al., 2023), and a machine learning-based predict-and-search algorithm Han et al. (2023). All algorithms were executed under a strict and identical time limit of 100 seconds to ensure a fair comparison based on the objective value (Obj) achieved, where a higher value indicates a better solution quality. The experimental results are summarized in Table 6.

Table 6: Experimental results of compared algorithms under a 100-second time limit in static scenarios.

| Algorithm | Instance (11, 40) | | Instance (12, 50) | |
|---|---|---|---|---|
| | Obj. | Time (s) | Obj. | Time (s) |
| ACO | 0.71770 | 20.03 | 0.76049 | 24.7487 |
| PS+Gurobi | 0.72644 | 6.2628 | 0.77548 | 66.1354 |
| **Ours+Gurobi** | **0.72644** | **4.1704** | **0.77551** | **62.0629** |

Given the characteristically rapid convergence properties of ACO metaheuristics, a supplementary ablation study was conducted. This experiment was designed to rigorously evaluate the comparative performance of the proposed NeuPath framework under an equivalently constrained, minimal time budget, thereby testing its efficacy in time-sensitive deployment scenarios.

NeuPath was executed under a strict time limit of 20 seconds. The comparative results, detailed in Table 7, demonstrate a consistent and significant performance advantage of the NeuPath framework over the ACO baseline across both tested problem instances.

Table 7: Comparative performance evaluation under a 20-second time constraint for NeuPath. Superior objective values are highlighted in bold.

| Algorithm | Instance (11, 40) | | Instance (12, 50) | |
|---|---|---|---|---|
| | Obj. | Time (s) | Obj. | Time (s) |
| ACO | 0.71770 | 20.03 | 0.76049 | 24.75 |
| NeuPath | **0.72617** | 13.95 | **0.774912** | 16.46 |

The empirical data reveals that NeuPath achieved markedly superior objective function values—0.72644 on instance (11,40) and 0.775 on instance (12,50)—compared to ACO, which attained 0.7177 and 0.76049, respectively, within the identical 20-second window. This performance delta underscores a fundamental strength of the proposed architecture.

This observed superiority under stringent time constraints can be attributed to two pivotal components of the NeuPath framework: first, the integrated GNN module demonstrates a remarkable capability to rapidly infer high-quality initial solutions, thereby bypassing the initial, often slow, exploratory phase characteristic of metaheuristics such as ACO; second, the trust-region search mechanism, guided by the GNN's probabilistic predictions, operates with high sample efficiency by concentrating the limited computational budget exclusively on the most promising regions of the solution space, thus avoiding wasteful exploration and ensuring more effective use of available resources.

In conclusion, this supplementary experiment robustly validates that the NeuPath framework not only matches but significantly exceeds the performance of a established heuristic, even when computational resources are severely limited. This confirms its potential for practical applications where both solution quality and time-to-solution are critical factors.

### *5.6. Parameter Analysis*

#### *5.6.1. Perparameter sensitivity analysis of focal loss*

We investigated the sensitivity of focal loss parameters on model performance. As shown in Figure 5, the class-balancing parameter $\alpha = 0.6$ yields

the highest objective value of 0.775480 with efficient training time (62.6s). Beyond $\alpha = 0.7$, performance stabilizes but training time increases by 5.6%.

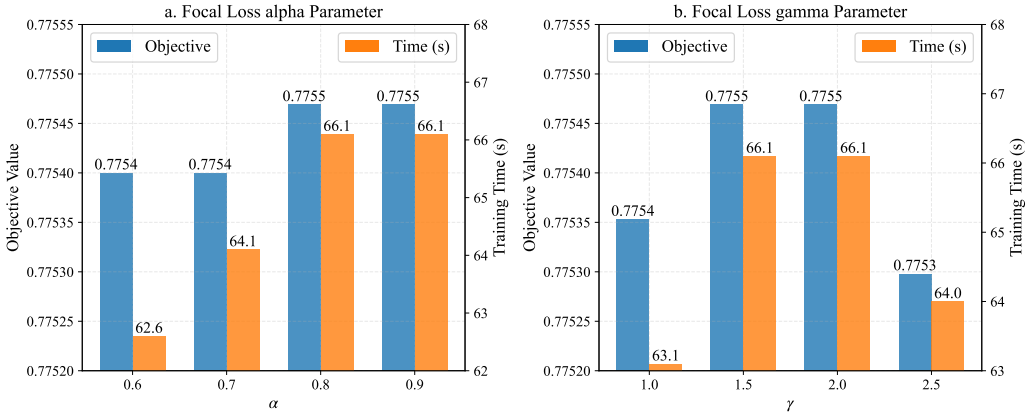


Figure 5 Sensitivity analysis results of Focal Loss parameters $\alpha$ and $\gamma$. The experiments were conducted on problem instances with scale $(12, 50)$ under a time limit of 100 seconds.

The focusing parameter $\gamma$ achieves optimal performance at $\gamma \in \{1.5, 2.0\}$ with objective value 0.775469. Increasing to $\gamma = 2.5$ causes performance degradation, suggesting excessive down-weighting of easy examples harms convergence. Both parameters demonstrate robustness within reasonable ranges, advantageous for practical applications.

### *5.6.2. Parameter sensitivity analysis of trust region*

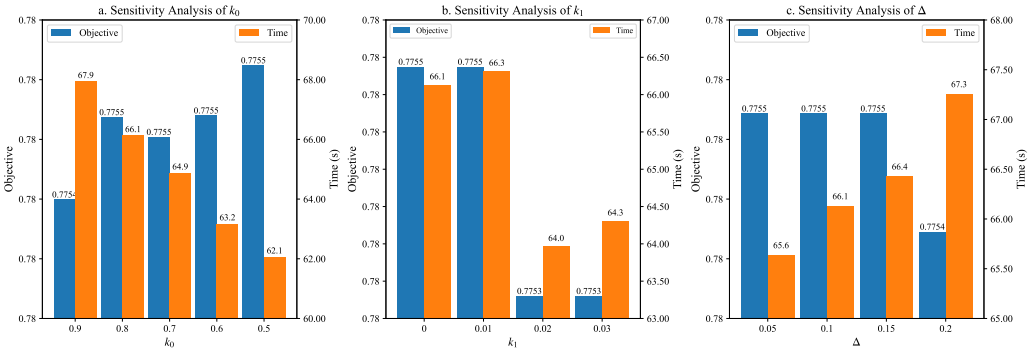


Figure 6 Sensitivity analysis results of trust region parameters on problem instances with scale $(12, 50)$ under 100s time limit.

We analyzed three trust-region parameters: $k_0$, $k_1$, and $\Delta$, using baseline values $k_0 = 0.8$, $k_1 = 0$, and $\Delta = 0.1$.

Parameter $k_0$ controls low-probability variable constraints. Decreasing $k_0$ from 0.9 to 0.5 improves objective value by 0.014% while reducing computation time by 8.7%. Parameter $k_1$ shows a threshold effect: objective values remain optimal for $k_1 \leq 0.01$ but degrade when $k_1 \geq 0.02$, indicating reliable GNN predictions for high-probability variables. The budget parameter $\Delta$ exhibits minimal sensitivity, maintaining stable performance for $\Delta \in [0.05, 0.15]$, with smaller values yielding better computational efficiency.

### *5.7. Ablation study*

The ablation study demonstrates the effectiveness of each proposed component in NeuPath. Across all three problem scales tested—$(11, 40)$, $(11, 50)$, and $(12, 50)$—several key findings emerge, as shown in Fig. 7:

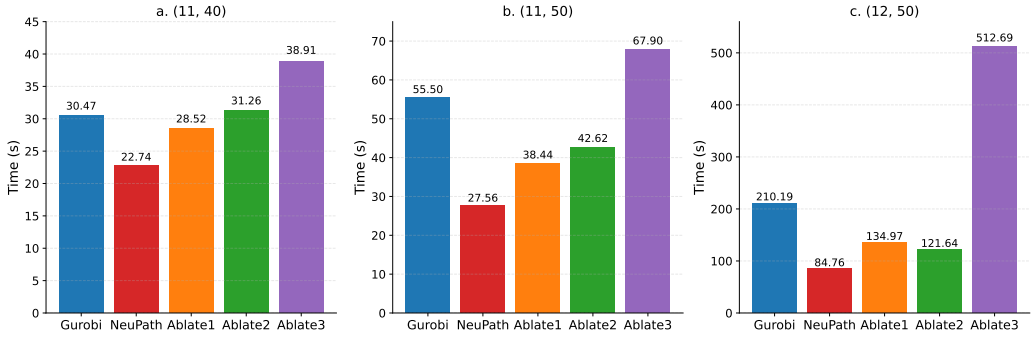


Figure 7 Ablation study results. Comparison of computational performance across different model configurations on three problem scales: (11, 40), (11, 50), and (12, 50). The evaluation includes our baseline solver (Gurobi), the complete NeuPath model, and three ablated versions: Ablate1 (replacing block aggregation with GNN from predict-and-search approach (Han et al., 2023)), Ablate2 (replacing focal loss with cross-entropyloss), and Ablate3 (replacing block trust region Search with conventional trust region search). The unit of time is seconds.

#### *5.7.1. Component analysis*

Replacing our block aggregation with GNN-based predict-and-search (Ablate1) increases computational time substantially (e.g., from 22.74s to 28.52s on scale (11, 40)), confirming our strategy's effectiveness in capturing structural relationships. Substituting focal loss with cross-entropy loss (Ablate2)

causes significant performance decline, validating its importance in addressing class imbalance between optimal and suboptimal solutions. Replacing block trust region search with conventional trust region search (Ablate3) results in marked performance degradation (e.g., from 38.91s to 67.90s on scale (11, 50)), highlighting the essential role of our block-structured approach.

*5.7.2. Component sensitivity and scale effects*

We quantify component sensitivity using relative performance degradation (RPD):

$$\text{RPD} = \frac{T_{\text{ablated}} - T_{\text{NeuPath}}}{T_{\text{NeuPath}}} \times 100\% \tag{25}$$

At scale (11, 50), block trust region search shows highest sensitivity (74.5% degradation), followed by block aggregation (25.4%) and focal loss (20%). Scale-dependent analysis reveals that block aggregation importance increases with problem complexity (from 25.4% at (11, 40) to 31.2% at (12, 50)), while block trust region maintains consistently high sensitivity (>70%) across all scales.

These findings establish a clear component ranking: block trust region search > block aggregation > focal loss, with all components proving necessary (minimum 20% impact). The exceptional sensitivity of block trust region search underscores that structure-aware exploration forms the cornerstone of NeuPath's efficiency.

## 6. Case Study

*6.1. Case description*

The accuracy of drift prediction and path planning depends on high-quality marine environmental data. We utilized comprehensive datasets from multiple authoritative sources.

Wind field data were obtained from ECMWF with global coverage at $0.125^\circ \times 0.125^\circ$ spatial resolution and variable temporal resolution: 3-hour intervals for the first four days and 6-hour intervals thereafter, covering a 10-day forecast period. Ocean current data came from HYCOM, offering $0.08^\circ \times 0.04^\circ$ spatial resolution and 1-hour temporal resolution over a 3-day forecast period. Data acquisition followed a systematic workflow: defining the geographical scope, submitting structured requests, and receiving NetCDF files (Ai et al., 2021). These underwent preprocessing including coordinate

transformation and temporal alignment before integration with the OpenDrift simulation framework (Dagestad et al., 2018).

To validate the practical applicability and robustness of the proposed NeuPath algorithm in real-world emergency scenarios, we conducted a comprehensive case study based on an actual person-overboard incident in the Zhuhai-Hong Kong-Macao maritime region. This case study serves to demonstrate the effectiveness of our approach in addressing the critical challenges of maritime search and rescue operations under realistic conditions.

The selected case involves a person-overboard incident that occurred on April 29, 2023, at 16:00 local time. Table 8 summarizes the key parameters of this emergency event.

Table 8: Detailed Information of the Maritime Incident Case

| **Parameter** | **Value** |
|---|---|
| Time of Incident | April 29, 2023, 16:00 |
| Incident Coordinates | 116°02'E, 21°17'N |
| Search Equipment | Unmanned Aerial Vehicle (UAV) |
| Wind Speed | 1.116 km/h |
| Current Speed | 4.08 km/h |
| Effective Search Radius | 0.1 nautical mile |

### *6.2. Data processing and visualization*

The marine environment data from the incident location were input into OpenDrift (Dagestad et al., 2018), a Lagrangian particle tracking framework, to simulate the drift trajectory of the person in water. Based on the drift predictions, we generated 4 time-segmented scenarios representing different phases of the search operation, each corresponding to hourly intervals following the initial incident. Taking the previous two sets of scenario data as examples, we visualize the POC distribution of the target search area, as shown in the figure 8. The search area was discretized into a grid structure based on the UAV's effective search radius of 0.1 nautical mile, creating a structured search space suitable for path planning optimization. The POC distribution was calculated for each grid cell, incorporating both drift uncertainty and detection probability factors.

### *6.3. Algorithm implementation*

To thoroughly evaluate the performance of NeuPath against the state-of-the-art commercial solver Gurobi, we conducted extensive experiments on

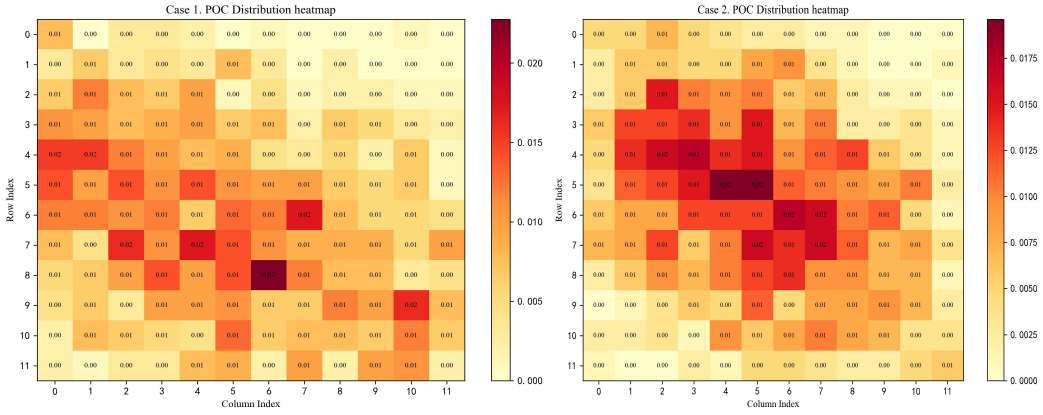


Figure 8 The POC distributions heatmap of the target search areas.The darker the color, the higher the probability value, and the greater the likelihood of the target's presence.

Scene data for the same accident at four different times from the Zhuhai-Hong Kong-Macao waters. Each algorithm was allocated a maximum computational time of 360 seconds per instance, reflecting realistic operational constraints in emergency response scenarios where timely decision-making is crucial. Throughout the experiments, we meticulously recorded two critical performance metrics: the best objective value achieved by each algorithm and the time required to reach this optimal solution.

### *6.4. Experimental results*

Table 9 presents the comprehensive performance comparison between Gurobi and NeuPath across the four test cases.

Table 9: Performance comparison on real-world maritime SAR cases.

| Case | Method | Best Objective | Time to Best (s) | Speedup |
|---|---|---|---|---|
| Case 1 | Gurobi | 0.5775 | 70 | 2.50× |
| | Ours+Gurobi | **0.5775** | **28** | |
| Case 2 | Gurobi | 0.6032 | 35 | 1.75× |
| | Ours+Gurobi | **0.6032** | **20** | |
| Case 3 | Gurobi | 0.6373 | 60 | 4.00× |
| | Ours+Gurobi | **0.6373** | **15** | |
| Case 4 | Gurobi | 0.6120 | 40 | 4.00× |
| | Ours+Gurobi | **0.6120** | **10** | |

A fundamental observation is that NeuPath achieves identical objective values to Gurobi across all test cases. This demonstrates that our neural-

enhanced approach maintains solution optimality while significantly reducing computational time, which is critical for successful rescue operations.

The comprehensive experimental evaluation on real-world maritime SAR cases conclusively demonstrates NeuPath's superiority in computational efficiency while maintaining optimal solution quality. The speedup translates to critical time savings in emergency response scenarios, making NeuPath a valuable advancement for maritime search and rescue operations.

## 7. Theoretical and practical implications

### *7.1. Theoretical implications*

This work makes several significant theoretical contributions to the intersection of machine learning and combinatorial optimization. First, we pioneer the representation of OSP as bipartite graphs, enabling the application of GNNs to capture complex structural patterns inherent in emergency search scenarios. This graph-theoretic formulation provides a principled framework for extracting problem features and facilitating dimensionality reduction, which represents a novel perspective in the OSP literature.

Second, we introduce a two-stage aggregation mechanism that effectively addresses the sparse and block-structured nature of OSP instances. The block-in and block-out aggregation strategy overcomes the limitations of traditional GNN architectures in handling highly structured optimization problems, providing theoretical insights into designing neural networks for sparse combinatorial problems.

Third, our integration of focal loss addresses the fundamental class imbalance challenge in optimization problems, where optimal solutions typically contain far more zero-valued variables than one-valued variables. This theoretical advancement demonstrates how specialized loss functions can significantly improve learning performance in imbalanced optimization contexts.

Finally, the block-wise trust-region scheme represents a novel hybrid approach that combines machine learning predictions with traditional optimization techniques. This framework provides theoretical guarantees while maintaining computational efficiency, bridging the gap between data-driven methods and exact optimization.

Furthermore, from a computational complexity standpoint, our framework offers a significant theoretical insight into managing NP-hardness. It delineates a clear demarcation between a fast, polynomial-time heuristic generation stage and the exponential-time exact search stage. Our approach

demonstrates how a scalable, linear-time investment can be used to effectively prune the search space of an exact solver. This model of "amortizing" the exponential cost of optimization with low-overhead, data-driven guidance represents a theoretically sound and scalable paradigm for tackling large-scale combinatorial problems.

### *7.2. Practical implications*

The proposed NeuPath framework demonstrates substantial practical benefits for real-world emergency response systems. Achieving average speedups of 2.6× over state-of-the-art exact solvers (Gurobi and SCIP) while maintaining solution quality makes this approach highly valuable for time-critical applications such as disaster response, search and rescue operations, and security missions.

It is well-suited to contemporary search and rescue (SAR) operations that employ intelligent unmanned systems, including autonomous underwater vehicles (AUVs), unmanned aerial vehicles (UAVs), and autonomous surface vessels (ASVs). Compared with conventional manned platforms, such systems provide increased mobility and operational flexibility. The adjacent-cell movement constraint incorporated in the model reflects the physical and operational requirement that, even for highly maneuverable unmanned platforms, movements typically follow continuous and energy-efficient trajectories rather than direct transitions between distant locations. This consideration is particularly relevant for battery-powered UAVs and fuel-limited ASVs, in which path efficiency directly affects mission duration and area coverage capability (Yang et al., 2020). The deterministic path-planning strategy described herein can be implemented on autonomous platforms with onboard computational resources, enabling optimal coverage decision-making without continuous human oversight. Such capability is of particular importance in environments where human deployment is infeasible or hazardous, including post-disaster areas, high sea states, and contaminated zones.

Moreover, this research establishes a foundation for developing adaptive intelligent systems that can learn from historical emergency scenarios and continuously improve their performance. The transferable patterns learned by our model can inform future emergency planning and resource allocation strategies, contributing to more effective disaster preparedness and response capabilities.

## 8. Conclusion

This paper introduces a hybrid learning-based optimization approach with two steps for the emergency path planning problem, addressing key challenges such as imbalanced solution vectors and sparse, block-structured search spaces. Experimental results show that our method outperforms professional exact solvers like Gurobi and SCIP across various problem sizes. On different scale instances, combining our model with Gurobi and SCIP reduces solution time by up to 2.6 times on average. The approach performs consistently well in both static and random target movement scenarios, highlighting its robustness. Moreover, our model generalizes effectively: it achieves significant speedups when trained on smaller problems and tested on larger ones, or when transferred across different search scenarios, indicating that it learns meaningful structural patterns.

In contrast to traditional heuristic methods, which often grapple with the trade-off between computational speed and solution quality and typically require instance-specific tuning, our NeuPath framework represents a paradigm shift. Its core advantage lies in its ability to learn the intrinsic, generalizable structural patterns of the optimization problem itself, rather than addressing each instance anew. This learned intelligence allows NeuPath to transcend the conventional speed-quality dichotomy, as evidenced by its dual capability to significantly accelerate exact solvers in finding optimal solutions and to deliver higher-quality solutions under tight time constraints in large-scale, complex scenarios where traditional methods falter. Crucially, the principled design of its components—validated through extensive ablation studies—ensures that this performance gain is not a "black-box" phenomenon but a result of a methodologically sound, hybrid approach. In essence, by synergistically integrating the predictive power of deep learning with the refinement of classical optimization, NeuPath offers a more robust, scalable, and intelligent paradigm for tackling time-critical search path problems.

In summary, this work demonstrates the potential of combining GNN with optimization to solve complex search tasks efficiently, paving the way for more intelligent and adaptive search strategies. Future work includes improving memory efficiency, extending the framework to general graph structures, and incorporating robustness to uncertain or adversarial target behavior. Promising directions also include integrating RL, extending to multi-agent settings, optimizing resource allocation, and applying the framework to other combinatorial problems like routing and scheduling for emergency.

## References


Ai, B., Jia, M., Xu, H., Xu, J., Wen, Z., Li, B., and Zhang, D. (2021). Coverage path planning for maritime search and rescue using reinforcement learning. *Ocean Engineering*, 241:110098.

Asfora, B. A., Banfi, J., and Campbell, M. (2020). Mixed-integer linear programming models for multi-robot non-adversarial search. *IEEE Robotics and Automation Letters*, 5(4):6805–6812.

Bengio, Y., Lodi, A., and Prouvost, A. (2021). Machine learning for combinatorial optimization: a methodological tour d'horizon. *European Journal of Operational Research*, 290(2):405–421.

Berger, J. and Lo, N. (2015). An innovative multi-agent search-and-rescue path planning approach. *Computers & Operations Research*, 53:24–31.

Cappart, Q., Chételat, D., Khalil, E. B., Lodi, A., Morris, C., and Veličković, P. (2023). Combinatorial optimization and reasoning with graph neural networks. *Journal of Machine Learning Research*, 24(130):1–61.

Chen, D.-S., Batson, R. G., and Dang, Y. (2011). *Applied integer programming: Modeling and solution*. John Wiley & Sons, Hoboken, NJ.

Dagestad, K.-F., Röhrs, J., Breivik, Ø., and Ådlandsvik, B. (2018). Opendrift v1. 0: a generic framework for trajectory modelling. *Geoscientific Model Development*, 11(4):1405–1420.

Ding, J., Zhang, C., Shen, L., Li, S., Wang, B., Xu, Y., and Song, L. (2020). Accelerating primal solution findings for mixed integer programs based on solution prediction. In *Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence*, pages 1452–1459. AAAI Press.

Eagle, J. N. (1984). The optimal search for a moving target when the search path is constrained. *Operations research*, 32(5):1107–1115.

Fave, F. M. D., Xu, Z., Rogers, A., and Jennings, N. R. (2010). Decentralised coordination of unmanned aerial vehicles for target search using the max-sum algorithm. pages 35–44.

Frost, J. (1999). Principles of search theory, part i: Detection. *Response, 17 (2)*, pages 1–7.

Furukawa, T., Durrant-Whyte, H. F., and Lavis, B. (2007). The element-based method-theory and its application to bayesian search and tracking. In *2007 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 2807–2812. IEEE.

Gasse, M., Chetelat, D., Ferroni, N., Charlin, L., and Lodi, A. (2019). Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc.

Griesbach, S. M., Hommelsheim, F., Klimm, M., and Schewior, K. (2025). Improved approximation algorithms for the expanding search problem. *ArXiv Preprint*.

Guo, Z., Li, Y., Liu, C., Ouyang, W., and Yan, J. (2024). Acm-milp: adaptive constraint modification via grouping and selection for hardness-preserving milp instance generation. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org.

Gupta, P., Gasse, M., Khalil, E. B., Mudigonda, P., Lodi, A., and Bengio, Y. (2020). Hybrid models for learning to branch. *Advances in Neural Information Processing Systems*, 33:18087–18097.

Han, Q., Yang, L., Chen, Q., Zhou, X., Zhang, D., Wang, A., Sun, R., and Luo, X. (2023). A gnn-guided predict-and-search framework for mixed-integer linear programming. *ArXiv Preprint*.

He, H., Daumé III, H., and Eisner, J. M. (2014). Learning to search in branch and bound algorithms. *Advances in Neural Information Processing Systems*, 27.

Heng, H., Ghazali, M. H. M., and Rahiman, W. (2024). Exploring the application of ant colony optimization in path planning for unmanned surface vehicles. *Ocean Engineering*, 311:118738.

Hermans, B., Leus, R., and Matuschke, J. (2022). Exact and approximation algorithms for the expanding search problem. *INFORMS Journal on Computing*, 34(1):281–296.

Hong, S.-P., Cho, S.-J., and Park, M.-J. (2009). A pseudo-polynomial heuristic for path-constrained discrete-time markovian-target search. *European Journal of Operational Research*, 193(2):351–364.

Huang, T., Ferber, A. M., Zharmagambetov, A., Tian, Y., and Dilkina, B. (2024). Contrastive predict-and-search for mixed integer linear programs. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 19757–19771. PMLR.

Khalil, E., Dai, H., Zhang, Y., Dilkina, B., and Song, L. (2017). Learning combinatorial optimization algorithms over graphs. 30.

Khalil, E. B., Le Bodic, P., Song, L., Nemhauser, G., and Dilkina, B. (2016). Learning to branch in mixed integer programming. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, pages 724–731. AAAI Press.

Khalil, E. B., Morris, C., and Lodi, A. (2022). Mip-gnn: A data-driven framework for guiding combinatorial solvers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 10219–10227.

Koopman, B. O. (1957). The theory of search: III. the optimum distribution of searching effort. *Operations Research*, 5(5):613–626.

Kratzke, T. M., Stone, L. D., and Frost, J. R. (2010). Search and rescue optimal planning system. In *2010 13th International Conference on Information Fusion*, pages 1–8. IEEE.

Kuang, Y., Wang, J., Liu, H., Zhu, F., Li, X., Zeng, J., HAO, J., Li, B., and Wu, F. (2024). Rethinking branching on exact combinatorial optimization solver: The first deep symbolic discovery framework. In *The Twelfth International Conference on Learning Representations*.

Lau, H., Huang, S., and Dissanayake, G. (2008). Discounted mean bound for the optimal searcher path problem with non-uniform travel times. *European Journal of Operational Research*, 190(2):383–397.

Li, X., Zhu, F., Zhen, H.-L., Luo, W., Lu, M., Huang, Y., Fan, Z., Zhou, Z., Kuang, Y., Wang, Z., Geng, Z., Li, Y., Liu, H., An, Z., Yang, M., Li, J., Wang, J., Yan, J., Sun, D., Zhong, T., Zhang, Y., Zeng, J., Yuan, M.,

Hao, J., Yao, J., and Mao, K. (2024). Machine learning insides optverse ai solver: Design principles and applications. *ArXiv Preprint*.

Li, Z., Li, G., Sadeghi, A., Sun, J., Wang, G., and Wang, J. (2025). Multi-vehicle cooperative persistent coverage for random target search. *IEEE Robotics and Automation Letters*, 10(7):6680–6687.

Liu, H., Wang, J., Geng, Z., Li, X., Zong, Y., Zhu, F., HAO, J., and Wu, F. (2025a). Apollo-MILP: An alternating prediction-correction neural solving framework for mixed-integer linear programming. In *The Thirteenth International Conference on Learning Representations*.

Liu, S., Cheng, H., Wang, Y., He, Y., and Fan, Changjun & Liu, Z. (2025b). Evopath: Evolutionary meta-path discovery with large language models for complex heterogeneous information networks. *Information Processing & Management*, 62(1):103920.

Ma, J., Hao, Z., and Sun, W. (2022). Enhancing sparrow search algorithm via multi-strategies for continuous optimization problems. *Information Processing & Management*, 59(2):102854.

Mao, W., Wu, J., Liu, H., Sui, Y., and Wang, X. (2024). Invariant graph learning meets information bottleneck for out-of-distribution generalization. *ArXiv Preprint*.

Mitsos, A., Chachuat, B., and Barton, P. I. (2009). McCormick-based relaxations of algorithms. *SIAM Journal on Optimization*, 20(2):573–601.

Morin, M., Abi-Zeid, I., and Quimper, C.-G. (2023). Ant colony optimization for path planning in search and rescue operations. *European Journal of Operational Research*, 305(1):53–63.

Morin, M., Papillon, A.-P., Abi-Zeid, I., Laviolette, F., and Quimper, C. G. (2012). Constraint programming for path planning with uncertainty: Solving the optimal search path problem. In *International conference on principles and practice of constraint programming*, pages 988–1003, Berlin, Heidelberg. Springer, Springer Berlin Heidelberg.

Nair, V., Bartunov, S., Gimeno, F., von Glehn, I., Lichocki, P., Lobov, I., O'Donoghue, B., Sonnerat, N., Tjandraatmadja, C., Wang, P., Addanki, R., Hapuarachchi, T., Keck, T., Keeling, J., Kohli, P., Ktena, I., Li, Y.,

Vinyals, O., and Zwols, Y. (2021). Solving mixed integer programs using neural networks.

Pal, A., Stojkoski, V., and Sandev, T. (2024). *Random Resetting in Search Problems*, pages 323–355. Springer Nature Switzerland, Cham.

Paulus, M. B. and Krause, A. (2023). Learning to dive in branch and bound. *Advances in Neural Information Processing Systems*, 36:34260–34277.

Paulus, M. B., Zarpellon, G., Krause, A., Charlin, L., and Maddison, C. (2022). Learning to cut by looking ahead: Cutting plane selection via imitation learning. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 17584–17600. PMLR.

Pu, T., Chen, C., Zeng, L., Liu, S., Sun, R., and Fan, C. (2024a). Solving combinatorial optimization problem over graph through qubo transformation and deep reinforcement learning. In *2024 IEEE International Conference on Data Mining (ICDM)*, pages 390–399. IEEE.

Pu, T., Fan, C., Shen, M., Lu, Y., Zeng, L., Nussinov, Z., Chen, C., and Liu, Z. (2024b). Exploratory combinatorial optimization problem solving via gauge transformation. In *2024 IEEE International Conference on Data Mining (ICDM)*, pages 821–826. IEEE.

Pu, T., Geng, Z., Liu, H., Liu, S., Wang, J., Zeng, L., Chen, C., and Fan, C. (2025). RoME: Domain-robust mixture-of-experts for MILP solution prediction across domains. In *Advances in Neural Information Processing Systems*, volume 39.

Qu, Q., Li, X., Zhou, Y., Zeng, J., Yuan, M., Wang, J., Lv, J., Liu, K., and Mao, K. (2022). An improved reinforcement learning algorithm for learning to branch. *ArXiv Preprint*.

Raap, M., Meyer-Nieberg, S., Pickl, S., and Zsifkovits, M. (2017). Aerial vehicle search-path optimization: A novel method for emergency operations. *Journal of Optimization Theory and Applications*, 172:965–983.

Raap, M., Preuß, M., and Meyer-Nieberg, S. (2019). Moving target search optimization – A literature review. *Computers & Operations Research*, 105:132–140.

Sato, H. and Royset, J. O. (2010). Path optimization for the resource-constrained searcher. *Naval Research Logistics*, 57(5):422–440.

Scavuzzo, L., Aardal, K., Lodi, A., and Yorke-Smith, N. (2024). Machine learning augmented branch and bound for mixed integer linear programming. *Mathematical Programming*, pages 1–44.

Scott, J. K., Stuber, M. D., and Barton, P. I. (2011). Generalized Mccormick relaxations. *Journal of Global Optimization*, 51(4):569–606.

Shibasaki, R. S., Rossi, A., and Gurevsky, E. (2024). A new upper bound based on dantzig-wolfe decomposition to maximize the stability radius of a simple assembly line under uncertainty. *European Journal of Operational Research*, 313(3):1015–1030.

Song, J., Yue, Y., Dilkina, B., et al. (2020). A general large neighborhood search framework for solving integer linear programs. 33:20012–20023.

Sonnerat, N., Wang, P., Ktena, I., Bartunov, S., and Nair, V. (2021). Learning a large neighborhood search algorithm for mixed integer programs. *ArXiv Preprint*.

Stewart, T. (1979). Search for a moving target when searcher motion is restricted. *Computers & Operations Research*, 6(3):129–140.

Stone, L. D., Royset, J. O., and Washburn, A. R. (2016). *Optimal Search for Moving Targets*. International Series in Operations Research & Management Science. Springer.

Stweart, T. J. (1980). *Experience with a Branch-and-Bound Algorithm for Constrained Searcher Motion*, pages 247–253. Springer US, Boston, MA.

Tang, Y., Agrawal, S., and Faenza, Y. (2020). Reinforcement learning for integer programming: Learning to cut. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 9367–9376. PMLR.

Trummel, K. and Weisinger, J. (1986). The complexity of the optimal searcher path problem. *Operations Research*, 34(2):324–327.

Wang, R., Wang, M., Zuo, L., Gong, Y., Lv, G., Zhao, Q., and Gao, H. (2025). The collaborative multi-target search of multiple bionic robotic fish based on distributed model predictive control. *Journal of Bionic Engineering*, 22(3):1194–1210.

Wang, R., Zhou, Z., Li, K., Zhang, T., Wang, L., Xu, X., and Liao, X. (2024). Learning to branch in combinatorial optimization with graph pointer networks. *IEEE/CAA Journal of Automatica Sinica*, 11(1):157–169.

Wang, Z., Li, X., Wang, J., Kuang, Y., Yuan, M., Zeng, J., Zhang, Y., and Wu, F. (2023). Learning cut selection for mixed-integer linear programming via hierarchical sequence model. In *The Eleventh International Conference on Learning Representations, ICLR 2023*.

Wu, B., Han, K., and Zhang, E. (2023). On the task assignment with group fairness for spatial crowdsourcing. *Information Processing & Management*, 60(2):103175.

Xu, X., Bao, X., Lu, X., Zhang, R., Chen, X., and Lu, G. (2023). An end-to-end deep generative approach with meta-learning optimization for zero-shot object classification. *Information Processing & Management*, 60(2):103233.

Yang, T., Jiang, Z., Sun, R., Cheng, N., and Feng, H. (2020). Maritime search and rescue based on group mobile computing for unmanned aerial vehicles and unmanned surface vehicles. *IEEE Transactions on Industrial Informatics*, 16(12):7700–7708.

Zarpellon, G., Jo, J., Lodi, A., and Bengio, Y. (2021). Parameterizing branch-and-bound search trees to learn branching policies. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 3931–3939.

Zeng, Q., Lin, L., Jiang, R., Huang, W., and Lin, D. (2025). Nnensleg: A novel approach for e-commerce payment fraud detection using ensemble learning and neural networks. *Information Processing & Management*, 62(1):103916.

Zhang, X. and Chen, D. (2023). Prepositioning network design for humanitarian relief purposes under correlated demand uncertainty. *Computers & Industrial Engineering*, 182:109365.